CO21BTECH11002
Aayush Kumar
ME5470 HW3 Report

Q1)

a, b)
The following functions are parallelized:

- grid, enforce_bcs, set_initial_condition
- get_error_norm_2d: private variables: j, local_diff, shared in reduction mode: norm_diff
- linsolve_hc2d_gs_adi: Each sweep along x direction is independent of each other, hence it was parallelized. Similarly for each sweep along y direction. Since each thread requires its own rhs_line and sol_line, they are initialized inside the for loop.
- timestep_BwdEuler: Initializing rhs, boundaries
- main(): Initializing 2D arrays.

In TDMA solver function (solve_tridiagonal), since the values it i-th steps depend on previous or next values, it can not be parallelized.

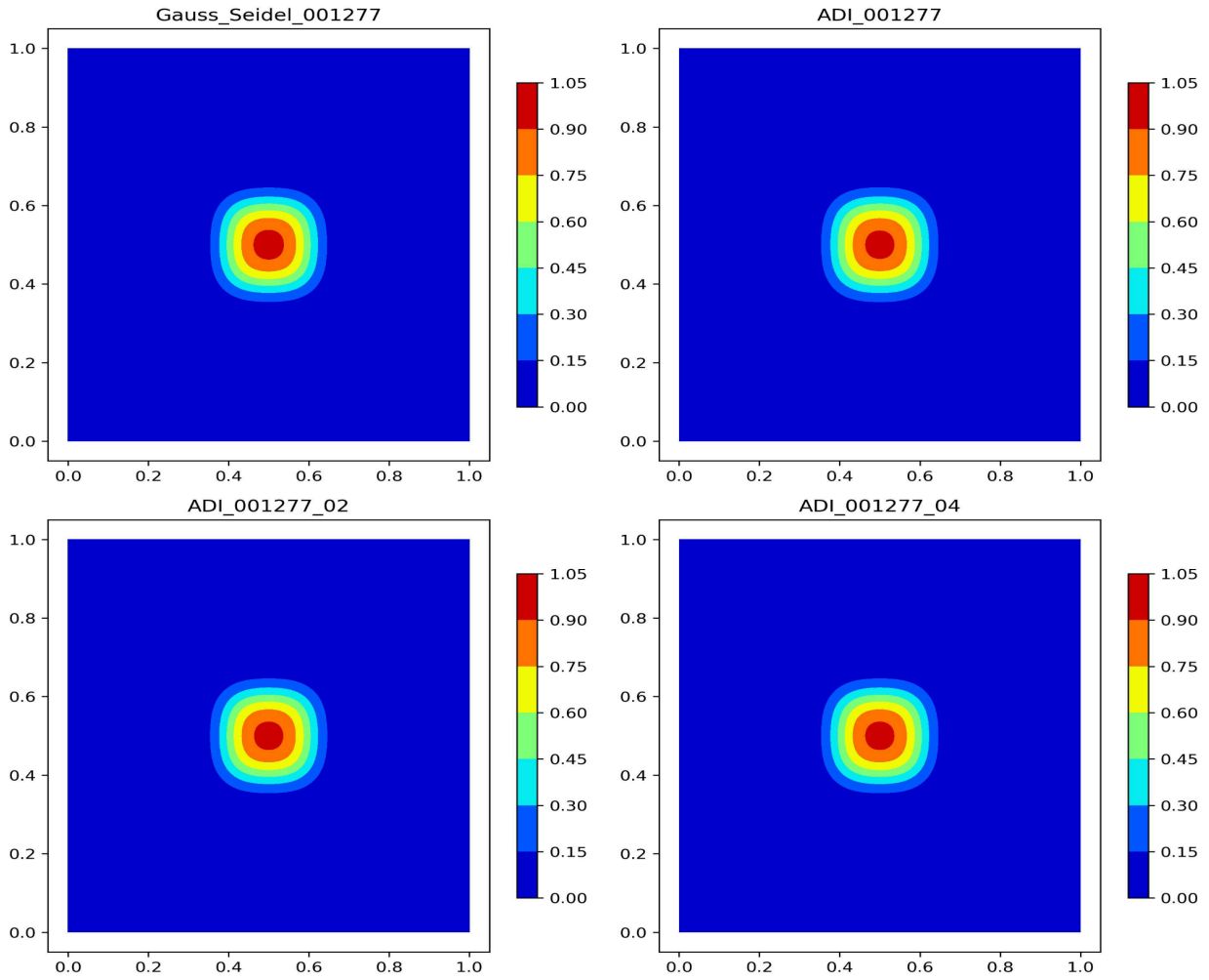

Figure 1: contours at t ≈ 0.001 for original serial code, the ADI serial code, ADI parallel code with p = 2 and p = 4.

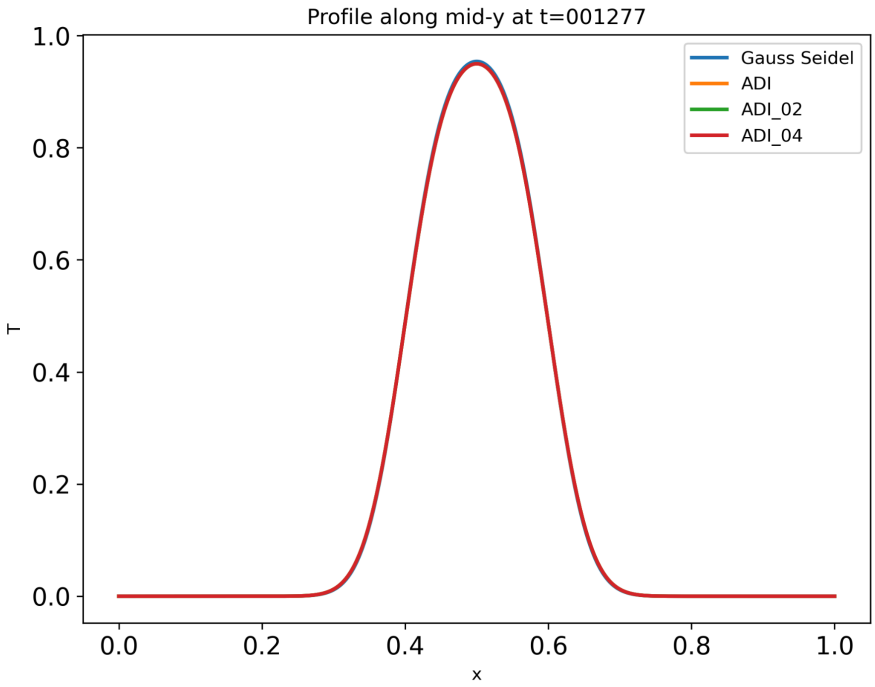

Figure 2: lines at mid-y at t ≈ 0.001 for original serial code, the ADI serial code, ADI parallel code with p = 2 and p = 4.

From both the plots, we can see that original serial code, the ADI serial code, ADI parallel code with p = 2 and p = 4 produce the same results.

c)

The following functions are parallelized:

- grid, enforce_bcs, set_initial_condition
- get_error_norm_2d: private variables: j, local_diff, shared in reduction mode: norm_diff
- linsolve_hc2d_gs_rb: Since each update for red points are independent of each other, the for loop is parallelized. Similarly for the black points.
- timestep_BwdEuler: Initializing rhs, boundaries
- main(): Initializing 2D arrays.

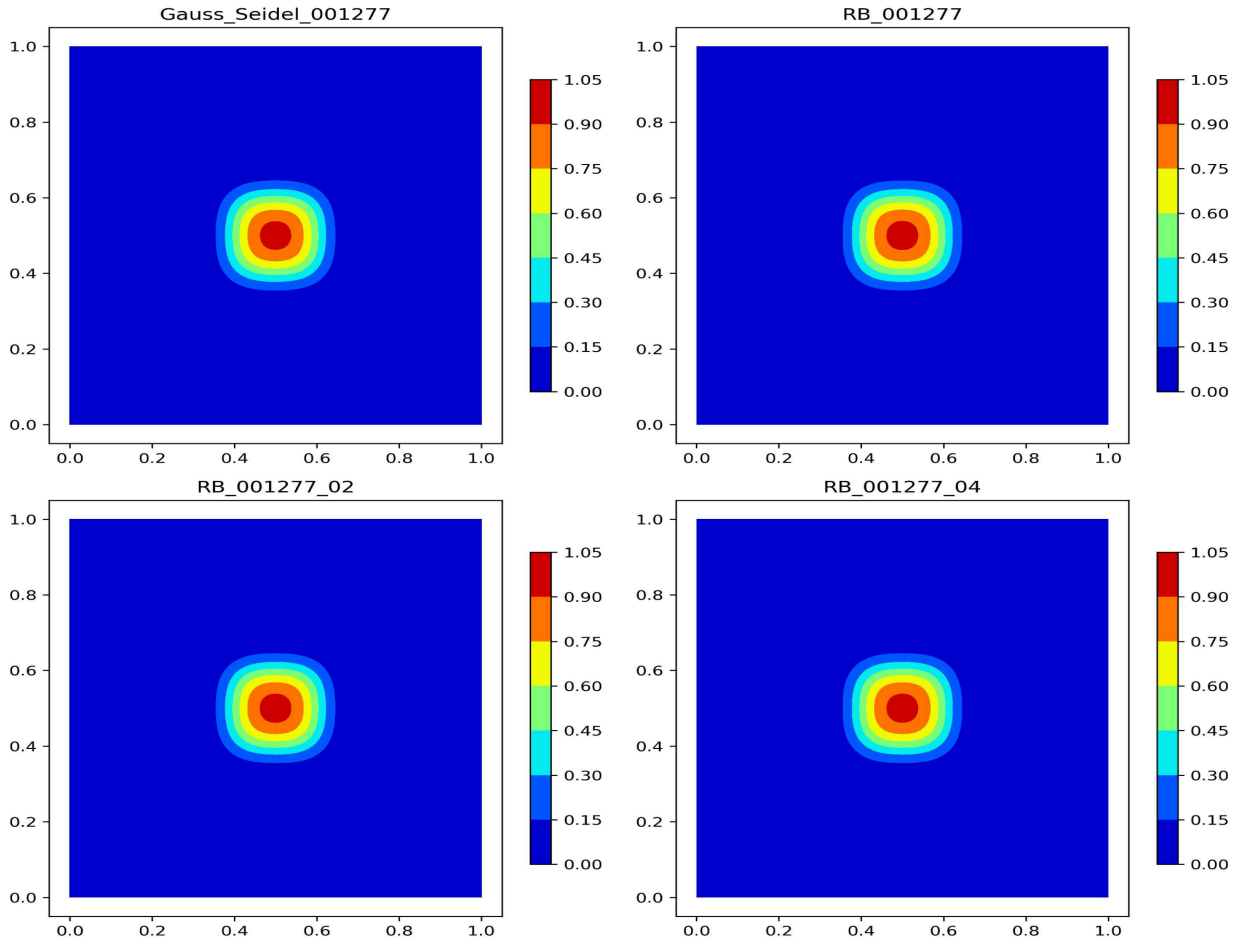

Figure 3: contours at t ≈ 0.001 for original serial code, the Red-Black (RB) serial code, RB parallel code with p = 2 and p = 4.

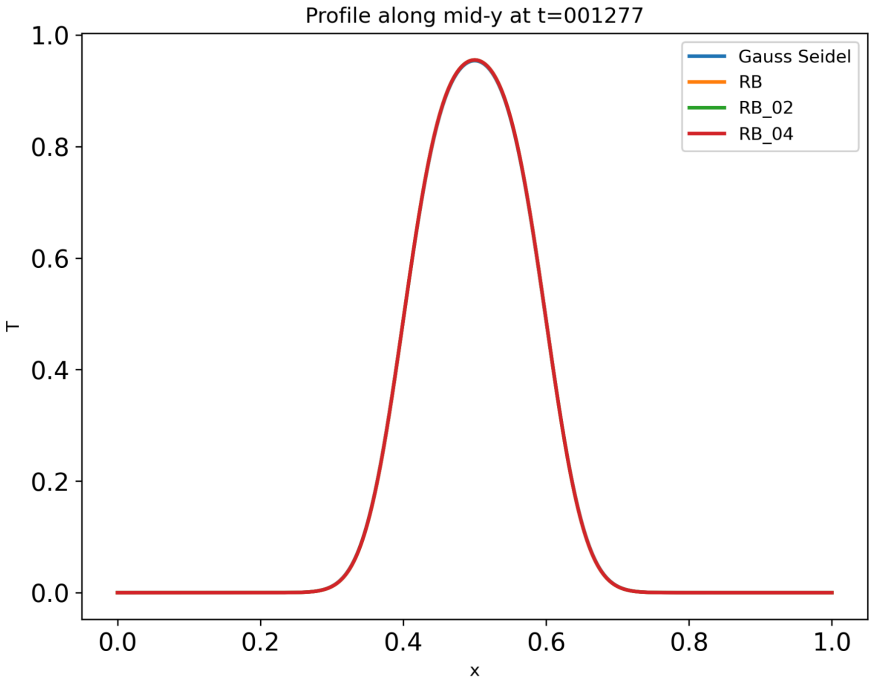

Figure 4: lines at mid-y at t ≈ 0.001 for original serial code, the Red-Black (RB) serial code, RB parallel code with p = 2 and p = 4.

From both the plots, we can see that original serial code, the Red-Black (RB) serial code, RB parallel code with p = 2 and p = 4 produce the same results.

Q2)
The code calculates the eigenvalue of a matrix by checking if a given vector satisfies the eigenvector equation A * v = λ * v. It reads the vector from a file, computes sum = A[i] * v for each row, and determines potential eigenvalues as eig = sum / v[j]. If all computed eigenvalues are the same (max_eig = min_eig), the vector is confirmed as an eigenvector, and the eigenvalue is printed. Otherwise, it prints "No."
The results are reduced to a root process, which checks for eigenvalue consistency.

The results are as follows for the given vectors:

Vector 1: Yes, eigenvalue = 0.333018
Vector 2: Yes, eigenvalue = 0.493142
Vector 3: Yes, eigenvalue = 0.939275
Vector 4: No